



Sardar Vallabhbhai National Institute of Technology (SVNIT),
Surat Department of Artificial Intelligence
Agentic AI (IA462), Semester VIII
U23AI052 Submission

Experiment 3

Tool-Using Agent — The Function-Calling Pattern

Aim

To implement the runtime half of LLM function calling — parsing a structured tool call, dispatching it to a registered Python function, and returning the result as an observation the agent can reason over.

Theory

A language model cannot compute reliably, does not know today's date and cannot reach external systems. Function calling closes that gap. The model is shown a set of tool schemas — for each tool a name, a description and its argument names and types — and instead of replying in prose it emits a structured request naming one tool and the arguments to call it with. The model itself executes nothing. The runtime parses the request, looks the name up in a tool registry, invokes the function and returns the value as an observation. That division is the whole point of the pattern: the model decides, the runtime executes, and every side effect therefore passes through code the developer controls.

This experiment implements the runtime side completely and scripts the model side, so the entire contract is visible without an API key or network access. The registry and dispatch function written here are reused in Experiment 4, where the same machinery sits behind the interleaved thought–action–observation steps of the ReAct loop.

Note the limitation visible in the sample run: dispatch trusts whatever it is handed, so a malformed JSON call or an unregistered tool name aborts the loop rather than producing a recoverable observation. Note also that the calculator is deliberately given a restricted evaluation environment, since a plain eval would run any expression the model chose to emit. Hardening the runtime against both is Exercise 1.

Table 1. Tool registry exposed to the model.

Tool name	Arguments (name: type)	What it returns
calculator	expression: string	Value of an arithmetic expression evaluated in a restricted namespace (sqrt, pow, pi)
days_between	d1: string, d2: string (ISO YYYY-MM-DD)	Whole number of days between the two dates
unit_convert	value: number, frm: string, to: string	The value converted between km/miles, kg/lb or °C/°F

Algorithm

1. Implement each tool as an ordinary Python function, register the functions in a name → function dictionary (the tool registry), and write a JSON schema for each one giving its name, description and argument types — the artifact a real model is shown.

TOOLS SHOWN TO MODEL:

```
[
  {
    "name": "calculator",
    "description": "Evaluate an arithmetic expression",
    "arguments": {
      "expression": "string"
    }
  },
  {
    "name": "days_between",
    "description": "Find the number of days between two dates",
    "arguments": {
      "d1": "string",
      "d2": "string"
    }
  },
  {
    "name": "unit_convert",
    "description": "Convert between supported units",
    "arguments": {
      "value": "number",
      "frm": "string",
      "to": "string"
    }
  }
]
```

2. Implement `dispatch()`: parse the JSON tool call, look the name up in the registry, invoke the function with the unpacked arguments and wrap the return value in a structured observation; then feed a scripted sequence of tool calls through it, printing each call with the observation it produced.

TOOL CALLS:

```
CALL: {"name": "calculator", "arguments": {"expression": "sqrt(25) + 2"}}
```

```
OBSERVATION: {'tool': 'calculator', 'observation': 7.0}
```

```
CALL: {"name": "days_between", "arguments": {"d1": "2026-09-01", "d2": "2026-09-10"}}
```

```
OBSERVATION: {'tool': 'days_between', 'observation': 9}
```

```
CALL: {"name": "unit_convert", "arguments": {"value": 10, "frm": "km", "to": "miles"}}
```

```
OBSERVATION: {'tool': 'unit_convert', 'observation': 6.21371}
```

Procedure and Result

Paste the snapshots of the steps followed and the output produced by the tool-using agent below, including the schemas presented to the model, every tool call issued and the observation returned for each one.

Observations

Comment on which side of the contract executes the tool and why that separation matters, on what happens to the loop when a call names a tool that is not in the registry, and on what the schema must contain for the model to be able to call a tool correctly without ever seeing its implementation.

Exercises

1. Add error handling to the runtime so that malformed JSON, an unknown tool name, a wrong argument name and an exception raised inside a tool each produce a structured error observation instead of crashing; demonstrate with one call of each kind.
2. Register a fourth tool that reads a CSV file and returns the mean of a named column. Write its schema and the tool call that invokes it, and report the observation returned.

```
import json

import math

from datetime import date

def calculator(expression):

    allowed = {

        "sqrt": math.sqrt,

        "pow": pow,

        "pi": math.pi

    }

    return eval(expression, {"__builtins__": {}}, allowed)
```

```
def days_between(d1, d2):  
    date1 = date.fromisoformat(d1)  
    date2 = date.fromisoformat(d2)  
  
    return abs((date2 - date1).days)  
  
def unit_convert(value, frm, to):  
  
    conversions = {  
        ("km", "miles"): lambda x: x * 0.621371,  
        ("miles", "km"): lambda x: x / 0.621371,  
  
        ("kg", "lb"): lambda x: x * 2.20462,  
        ("lb", "kg"): lambda x: x / 2.20462,  
  
        ("C", "F"): lambda x: (x * 9 / 5) + 32,  
        ("F", "C"): lambda x: (x - 32) * 5 / 9  
    }  
  
    return conversions[(frm, to)](value)  
  
# Maps the tool name to the actual Python function  
tools = {  
    "calculator": calculator,  
    "days_between": days_between,  
    "unit_convert": unit_convert  
}
```

```
schemas = [  
    {  
        "name": "calculator",  
        "description": "Evaluate an arithmetic expression",  
        "arguments": {  
            "expression": "string"  
        }  
    },  
    {  
        "name": "days_between",  
        "description": "Find the number of days between two dates",  
        "arguments": {  
            "d1": "string",  
            "d2": "string"  
        }  
    },  
    {  
        "name": "unit_convert",  
        "description": "Convert between supported units",  
        "arguments": {  
            "value": "number",  
            "frm": "string",  
            "to": "string"  
        }  
    }  
]
```

```
def dispatch(tool_call):
```

```

# Convert JSON string into Python dictionary
call = json.loads(tool_call)

# Get tool name
name = call["name"]

# Get arguments
arguments = call["arguments"]

# Find the function in our registry
function = tools[name]

# Execute the function
result = function(**arguments)

# Return observation
return {
    "tool": name,
    "observation": result
}

print("TOOLS SHOWN TO MODEL:")
print(json.dumps(schemas, indent=2))

print("\nTOOL CALLS:\n")

calls = [
    '{"name": "calculator", "arguments": {"expression": "sqrt(25) + 2"}}',

```

```
    '{"name": "days_between", "arguments": {"d1": "2026-09-01", "d2":  
    "2026-09-10"}}',  
  
    '{"name": "unit_convert", "arguments": {"value": 10, "frm": "km",  
    "to": "miles"}}'  
]  
  
for call in calls:  
  
    print("CALL:", call)  
  
    observation = dispatch(call)  
  
    print("OBSERVATION:", observation)  
    print()
```

